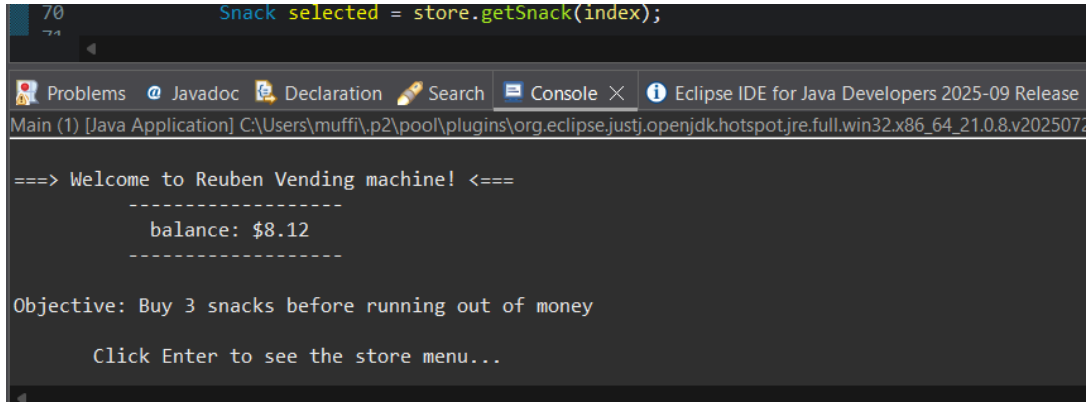
Reu Reu Vending Machine Simulator By Reuben Thomas

My project is a text-based vending machine.

Simulate a vending machine experience in Java

- Console-based menu
- Balance tracking & penalties
- Objective: Buy 3 snacks before running out of money



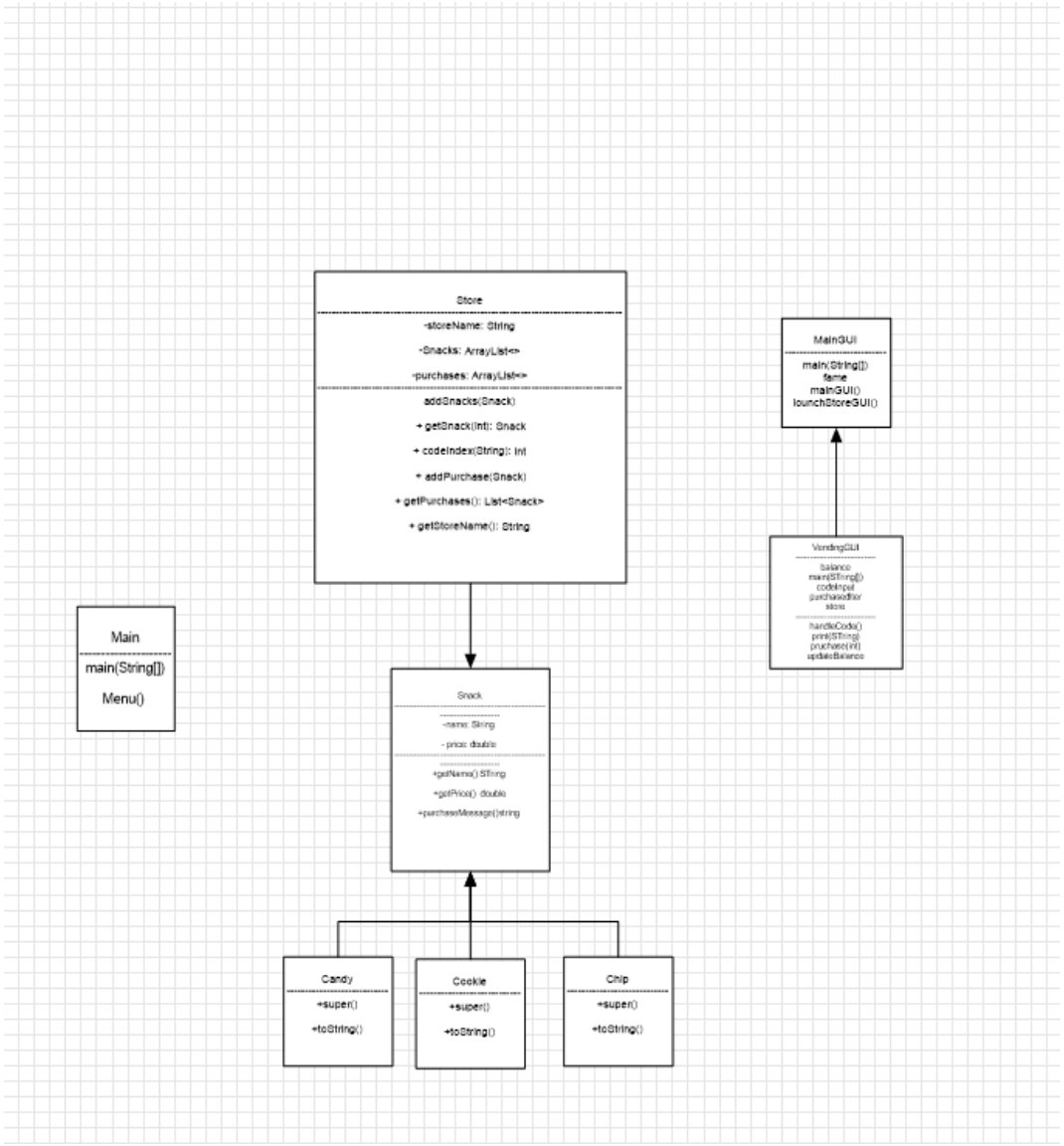
Structure

- Main class: Handles game loop, balance, purchases
- Store class: Manages snacks, menu, purchase history
- Snack (base class)

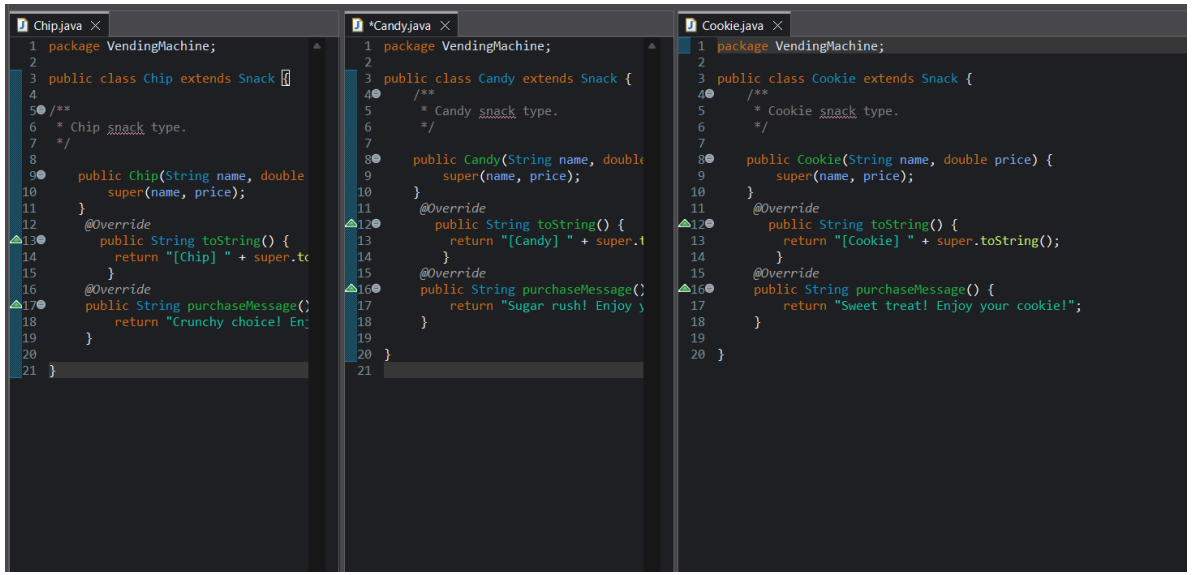
Candy, Cookie, Chip (subclasses with custom messages)

- GUI classes: VendingGUI, MainGUI

UML Diagram


Store
-storeName: String
-Snacks: ArrayList<>
-purchases: ArrayList<>
addSnacks(Snack)
+ getSnack(Int): Snack
+ codeIndex(String): Int
+ addPurchase(Snack)
+ getPurchases(): List<Snack>
+ getStoreName(): String
MainGUI
main(String[])
frame
mainGUI()
launchStoreGUI()
VendingGUI
Main
main(String[])
Menu()
Snack
-name: String
- price: double
+getName(): STring
+getPrice(): double
+purchaseMessage():string
Candy
+super()
+toString()
Cookie
+super()
+toString()
Chip
+super()
+toString()

Inheritance and polymorphism



- Inheritance: my base class (Snack) I extends 3 subclass to it. " Snack(super(name,price));"

- Polymorphism: my method name (purchaseMessage) is different depending on the object type

```
 8  */
 9
10
11  public class Store {
12
13      private String storeName;
14      private ArrayList<Snack>Snacks;
15      private ArrayList<Snack> purchases;
16
```

substantive collections

- My ArrayList<snacks>Snacks hold all the available snacks in the vending machine. This make it a substantive collection because it store multiple objects of different subclass

- ArrayList<snacks>purchase records the history, print at end.

Exception handling

```
            System.out.println("");
            System.out.println("       Click Enter to see the store menu...");
            input.nextLine();
        //  store.showMenu();

         Menu();
         try {

            System.out.println("\nEnter item code or X to exit: ");
            String code = input.nextLine();

            //String code = null;
            if(code.equalsIgnoreCase("X")) {
                System.out.println("Exiting.....");
                break;
            }

            int index = store.codeIndex(code);
            Snack selected = store.getSnack(index);
```

```
        }

    }catch (IllegalArgumentException e) {
            System.out.println("Error: " + e.getMessage());
       } catch (RuntimeException e) {
             System.out.println("Error: " + e.getMessage());
            running = false;
       }
```

- Keeps program running
- Improves reliability
- Provides clear error messages